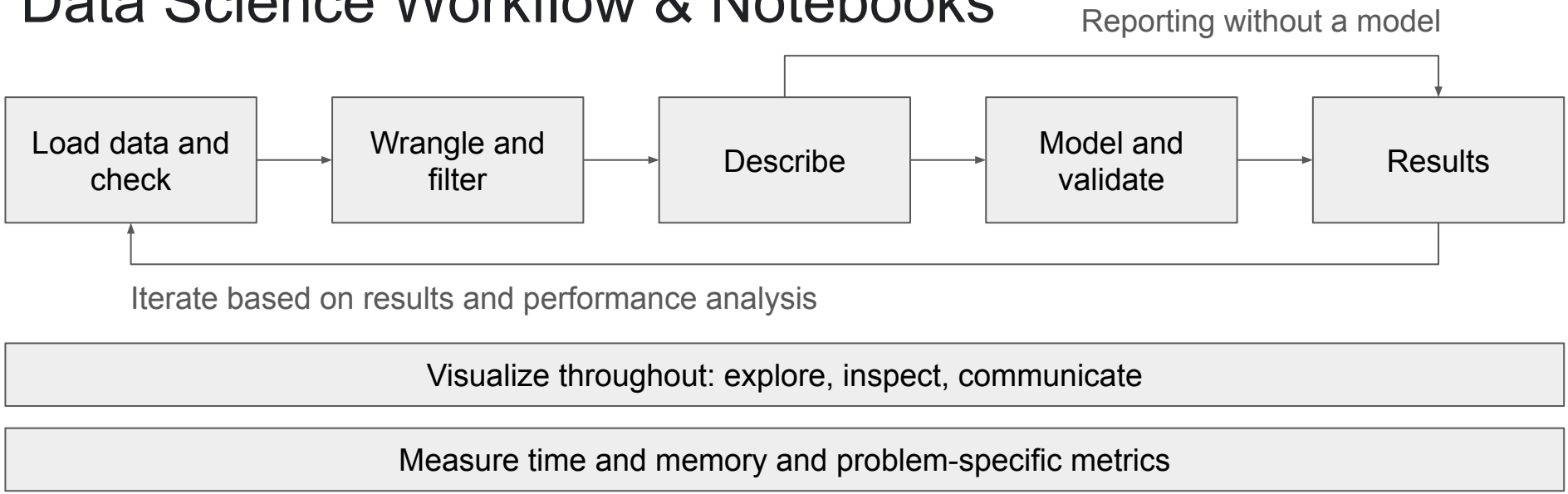
Operating Systems and Parallel Computing

Lecture 8

Prof. Matteo Turilli

Data Science Workflow & Notebooks



- Comments explain unfamiliar Python and what each loop keeps between iterations.
- Explanations connect the operation, the observation, and the reason for the result.
- Each notebook maps its experiments to the lecture and to a data-science workflow.
- Notebook 6 separates Pandas storage (6A) from process memory (6B).
- Experiments retain their scope and include checks of their results.

questions answered by the notebooks

- Each notebook asks a systems question about the same sales-analysis project.
- Dataset for the project is in our container: `data/sales_2025_sample.csv`
- `05_syscall_cost`:
  - Main question: Which repeated file operations can we group together?
  - Data Science workflow: Loading and saving
- `06_memory_panda`:
  - Main question: How much storage do the columns need?
  - Data Science workflow: Loading and preparation
- `06_memory_process`:
  - Main question: What memory does the whole Python process use?
  - Data Science workflow: Throughout the project
- `07_memory_budget`:
  - Main question: What must we keep while processing more rows?
  - Data Science workflow: Loading, preparation and description

Mapping Notebooks to Slides

Notebook	Lecture Slides	Connection
`05_syscall_cost`	L5: 8–11, 14–16, 21–22	User/kernel boundary: fewer repeated file operations leads to better performance
`06_memory_panda`	L6: 18–19	Column representations: fewer bytes per row, same values.
`06_memory_process`	L6: 7–8, 13–15, 19	Address spaces and pages: allocation can precede physical use.
`07_memory_budget`	L7: 3–10, 13–16	Memory pressure and OOM: keep less input while preserving the answer.

- All notebooks refer to Lecture 1 stack: application code, Python runtime and libraries, operating system, and hardware
- Each notebook connect a decision in our code to the work needed to carry it out

5: Request size and Python buffering

- Using `os.read()`, we vary the request from 1 byte to 1 MiB while reading the same 6 MB file, so the data stay constant.
- One-byte requests need 6,013,655 Python calls and 1,730.384 ms, while 64 KiB requests need 93 calls and 0.245 ms.
- Fewer iterations repeat less Python and kernel work, which explains the large gain without isolating the cost of a syscall.
- Further gains can shrink or reverse because the same file must still be copied, and larger chunks need more temporary memory.
- Using `open()` for one-byte reads is 14.6 times faster here because its buffer serves many requests without entering the kernel.
- Request size sets how many bytes we receive, while Python's buffer has a separate capacity of 128 KiB in this container.

5: Many files or one combined file

- We load the same 100,000 records from 1,000 small CSV files and from one combined file, then check that both return equivalent data.
- One combined file avoids repeating file opening and CSV-reader setup, which can otherwise dominate many small loads.
- In this run, the combined file loads in 0.06 s rather than 0.368 s, making each load about 6.1 times faster.
- Combining the files first costs 0.749 s, so the 0.308 s saved per later load must repay that preparation.
- Three later loads recover the preparation cost, while a single load would not justify combining the files in this run.
- Consolidation therefore helps when we revisit the data often enough, e.g., while adjusting an analysis or rerunning a report.

5: Saving results in batches

- We save the same 5,000 rows using three strategies, so differences in time come from how we organise the writing.
- Reopening for every row takes 0.566 s because each row repeats opening, CSV conversion, writing and closing.
- Keeping the file open reduces the time to 0.423 s, while 5,000 calls to `to_csv()` still repeat slicing and CSV formatting.
- Keeping it open and writing 500 rows per batch needs only ten `to_csv()` calls and takes about 0.004 s.
- All three outputs contain identical bytes, showing that batching reduces repeated work while preserving the saved result.
- Batches give the largest saving here, while larger batches can need more memory and delay when new results are saved.

6A: Measuring the loaded table

- Loading the CSV turns 100,000 rows and nine columns into Pandas arrays and other objects, so file size does not tell us their memory cost.
- `memory_usage()` estimates storage for the columns and row index, giving us a measure of the table rather than the whole process.
- For `object` columns, a shallow estimate can miss the Python objects holding the values, while `deep=True` includes those objects.
- Our text columns use Arrow, a library that stores column data in buffers whose bytes are already counted in the shallow estimate.
- Both estimates therefore report 10.56 MiB, and `ArrowStringArray` confirms the text representation used in this run.
- The full analysis needs more memory because Python, libraries, other tables and temporary calculations also occupy RAM.

6A: Categories can save or cost memory

- A category stores each distinct label once and gives every row an integer code, so repeated labels can use less memory.
- The `region` column has only six labels across 100,000 rows, and its storage falls from 1.301 to 0.096 MiB.
- The `customer_id` column has 60,417 distinct labels, so it must keep a much larger label set as well as 100,000 row codes.
- Its labels and codes use about 1.188 MiB, but a lookup structure adds 2.016 MiB to help Pandas find each label.
- The complete category therefore uses 3.204 MiB rather than 1.335 MiB as text, making it larger despite repeated customer IDs.
- A category helps only when saved label storage outweighs its codes and lookup structures, so measure and verify before converting.

6A: Choosing and checking column types

- We check ranges and missing entries before choosing smaller integer types because those types have limits on the values they can store.
- `order_id` values from 1 to 100,000 fit `int32`, while `quantity` values from 1 to 20 fit `int16`.
- Repeated labels, including `product_id`, become categories, while date text becomes date values to reduce storage.
- Prices stay `float64` to avoid losing more precision, and `customer_id` stays text because its category would cost more memory.
- The same nine-column table now uses 4.27 MiB instead of 10.56 MiB, leaving more room for joins, calculations or plots.
- We accept the change after records and regional totals agree with the reference, and we recheck the assumptions for each new file.

6B: The table and the Python process

- VIRT describes the process's virtual address space, while RSS reports the memory currently resident in RAM.
- Loading the CSV raises VIRT from 913.0 to 1,943.1 MiB, but that increase does not mean the process uses that much more RAM.
- RSS rises from 157.0 to 187.3 MiB, adding about 30.3 MiB even though the loaded table is estimated at 10.56 MiB.
- Python may keep memory for reuse after parsing, and RSS also includes the runtime, imported libraries and other objects.
- Because the reader creates temporary objects, readings taken only before and after loading can miss its highest memory use.
- These readings describe the process, so use the container's memory limit and other running work when planning the full analysis.

6B: Requesting, using and releasing memory

- The NumPy experiment separates requesting memory from writing to it: loading a CSV combines both operations.
- `np.zeros()` creates a zero-valued array needing 190.7 MiB of element storage. In this run, VIRT grows by that amount.
- RSS stays at 187.7 MiB after allocation: the new address range does not yet need that much additional RAM.
- Writing across the array's pages raises RSS to 378.4 MiB. VIRT stays unchanged because the address range already exists.
- `del big` removes the reference and RSS falls to 187.8 MiB, but it may not always fall: Python and its libraries can keep released memory for later objects.
- Note: Successful allocation does not prove a later calculation will fit because writes and temporary objects can increase memory demand.

7: Establishing the regional answer

- Our question is how much `revenue` each `region` generated in 2025. All 100,000 sample transactions belong to that year.
- For each transaction, `revenue = quantity * unit_price`. Summing by `region` gives the six totals needed for the report.
- The sample fits in memory, so we first use the complete table to establish a reference answer.
- Each alternative must preserve the `region` labels and agree within half a cent: using less memory is useful only if the answer holds.
- We then change what is retained, i.e., fewer columns, smaller representations, chunks or individual rows.
- This connects loading and preparation to reporting: the required result determines which data must remain in memory.

7: Selecting the required columns

- The report needs `region`, `quantity` and `unit_price`. The other six input columns do not contribute to this answer.
- `usecols` tells `pd.read_csv()` to retain those three inputs. Note that the full CSV still has to be scanned, so this does not remove all reading work.
- After calculating `revenue`, we keep only `region` and `revenue` for each transaction, while `quantity` and price have served their purpose.
- This final two-column table uses 2.06 MiB, compared with 10.56 MiB for the original nine-column table.
- All 100,000 rows still contribute and the regional totals agree: we saved storage by removing fields, without dropping transactions.
- The 2.06 MiB describes the retained table. Parsing buffers and temporary calculations also need memory while it is built.

7: Keeping all columns in less space

- If later questions need all nine columns, we can change their representations instead of dropping information (as we learned in notebook 6A).
- Range and missing-value checks let us use smaller integers safely for this sample.
- Repeated labels use `category` while prices stay `float64`. These choices reduce storage while keeping the values needed.
- The nine-column table falls from 10.56 to 4.86 MiB, before adding `revenue`. All six regional totals still agree.
- This is larger than 6A's 4.27 MiB because the dtype choices differ, including how `product_id` is stored.
- We should combine dtypes with column selection when useful: fewer columns and fewer bytes per value reduce different costs.

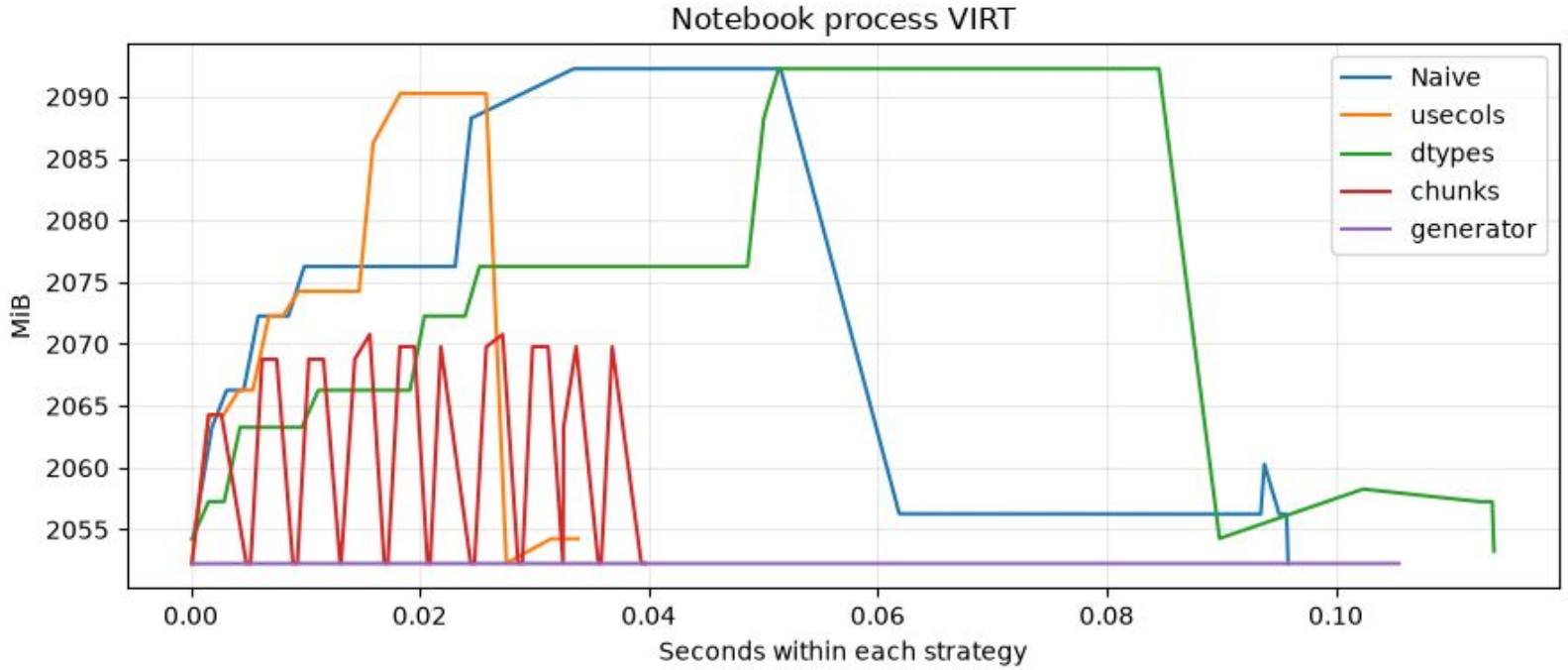
7: Processing chunks and retaining totals

- With `CHUNK_ROWS = 10_000`, the sample arrives in ten chunks. In this way, we do not load all ten into a list.
- Each chunk contributes regional subtotals. The `totals` dictionary survives between chunks, so earlier contributions are preserved.
- Once a chunk has contributed, its rows are no longer needed for this sum. Removing their references allows storage to be reused.
- The largest measured chunk uses 0.36 MiB, including `revenue`. The final six `totals` agree with the full-table reference.
- With six fixed regions, more rows mean more iterations. Millions of customer groups would make the retained dictionary grow.
- Small chunks repeat work more often, while large chunks need more memory. Note that `CHUNK_ROWS` counts rows, not the byte requests from notebook 5.

7: Supplying one row at a time

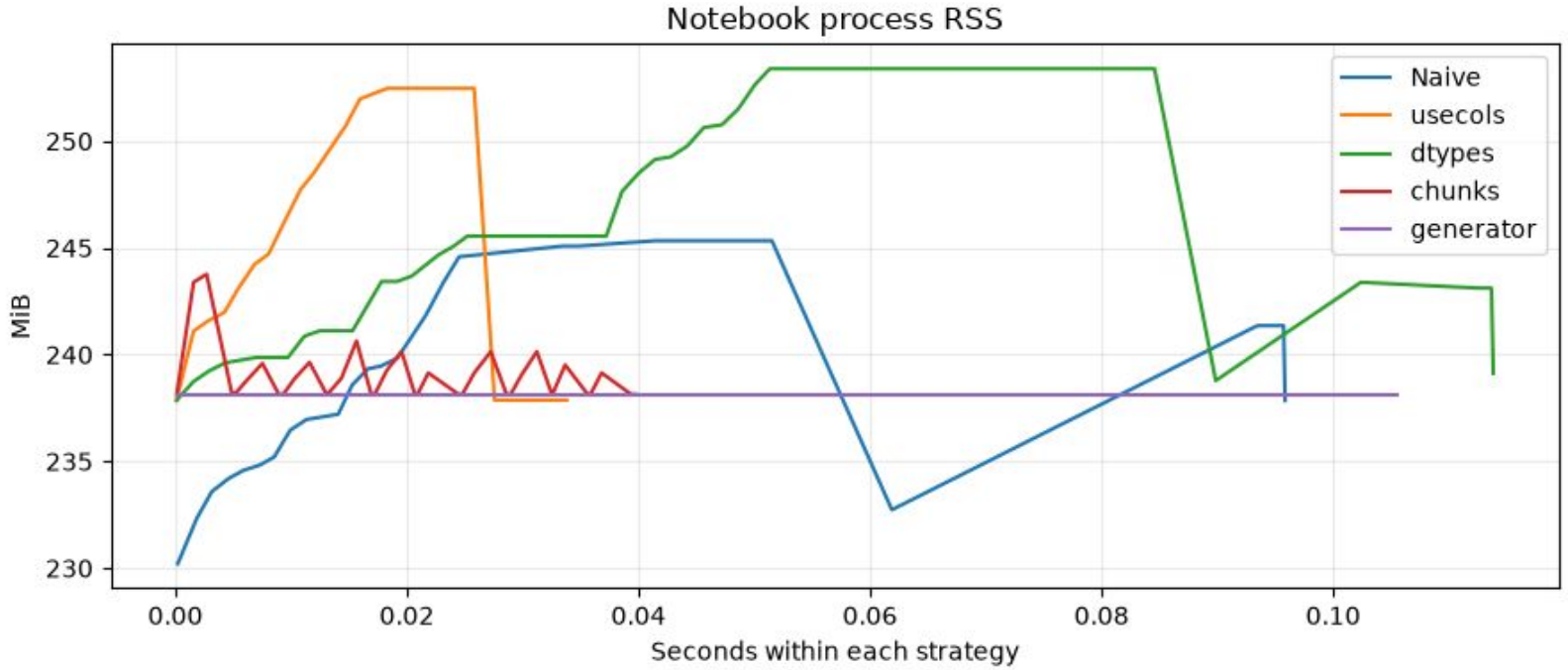
- `revenues_by_region()` supplies a `region` and its `revenue` for each transaction, without first building a full table or list.
- `yield` supplies one pair and pauses the function. The consuming loop requests the next pair when it is ready.
- The consumer adds each contribution to a persistent dictionary. In this way, only the current row and running totals are needed for the sum.
- Streaming limits retained input but it does not eliminate memory use. Reader buffers and generator state still occupy memory.
- The six totals pass the half-cent check. The largest difference is 0.00000006, reflecting small floating-point differences.
- Python converts and processes every row separately. This can take longer than Pandas operations over a whole chunk.

7: Reading VIRT: virtual address space



- VIRT measures address space. A higher value does not mean more RAM use.
- Flat lines show steady samples, and brief peaks can occur between readings.

7: Reading RSS: memory held in RAM



- The runs were sequential and each line follows one strategy from its own start.
- Compare starting levels before rises: RSS includes earlier tables and libraries.

7: Table storage and process memory

- A DataFrame estimate describes one table. RSS includes the Python process, its libraries and other resident allocations.
- The saved sampled maximum is 245.33 MiB for the full load and 252.47 MiB for `usecols`, despite its smaller retained table.
- All strategies run in the same process, where earlier tables remain. Allocators can also keep released memory for reuse.
- A later strategy may therefore start from a different baseline; these maxima do not rank each strategy's memory requirement.
- Sampling can miss brief peaks. The largest observed value is evidence about this run, not a guaranteed upper bound.
- To compare process peaks fairly, run each strategy in a fresh process with the same setup and measurement method.

Exercises

- Trace two chunks: Iberia contributes 20, then 35. The retained total must become 55, even after the first chunk is discarded.
- Locate `totals = {}` outside the loop. Moving it inside would erase earlier contributions and change the answer.
- Follow one update from the chunk's subtotal to the dictionary; explain which values must survive the next iteration.
- Change `CHUNK_ROWS` to `1_000`, then `50_000`. Predict the chunk count, largest chunk storage and elapsed time before running.
- Run `check_totals()` before comparing resources. A faster or smaller calculation is not useful if it loses or counts rows twice.
- Use the measurements to justify a chunk size for this report, and explain why another question might need different retained data.